

outputs even in the zero-shot setting, with limited additional benefit observed from few-shot examples, while the few-shot examples can indeed provide some performance improvement.

We find that the conclusions mentioned in main paper have still holds. For example, the performance also significantly decreases as the length increases or the tasks and representations become unfamiliar (like Add-Fraction, Add-Scientific or floordiv). And the performance of digit-related tasks are still unsatisfying.

Table 10: Few-shot performance on selected tasks

Model	Add Int				Max Float				Floordiv Int			
	S	M	L	XL	X	M	L	XL	X	M	L	XL
Llama-2-7b-hf-5-shot	0.61	0.12	0.00	0.00	0.68	0.55	0.49	0.43	0.04	0.01	0.01	0.00
Llama-2-7b-hf	0.74	0.11	0.00	0.00	0.44	0.47	0.28	0.15	0.04	0.01	0.00	0.00
Llama-3.1-8B-5-shot	0.94	0.41	0.10	0.01	0.88	0.81	0.63	0.54	0.23	0.02	0.01	0.00
Llama-3.1-8B	0.95	0.38	0.06	0.02	0.70	0.57	0.41	0.36	0.19	0.01	0.01	0.01
Qwen2-7B-5-shot	0.83	0.82	0.37	0.04	1.00	0.98	0.81	0.64	0.28	0.08	0.03	0.01
Qwen2-7B	0.93	0.70	0.23	0.03	0.68	0.72	0.55	0.43	0.22	0.05	0.01	0.02

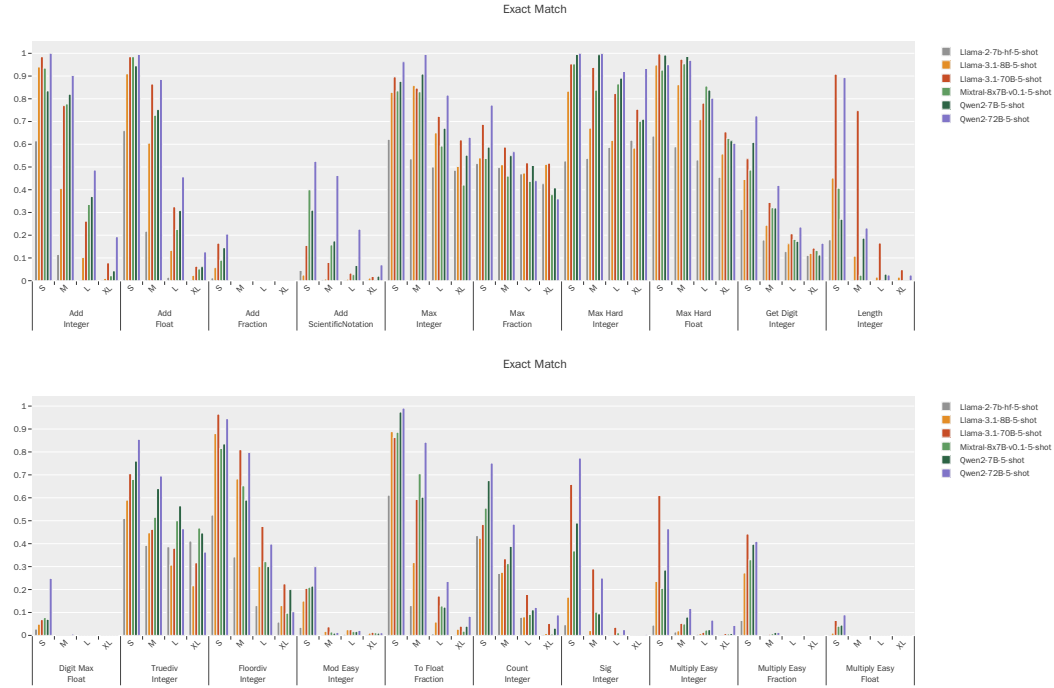


Figure 8: Parts of performance of state-of-the-art LLMs on NUPA benchmark with 5-shot examples.

A.4 TOKENIZER, PE AND DATA FORMATS

A.4.1 EXPERIMENT DETAILS

We train several models to test the effectiveness of tokenizers, PEs and data formats. Unless otherwise mentioned, our model architecture uses the Llama-3.1 architecture (Decoder-only Transformers with causal masking, autoregressive generation, and RoPE as the default PEs). We modify the layer numbers, hidden size and the number of heads to change the parameter size of models. See Table 11. We keep all hyperparameters, except model size, consistent with the original Llama setup in the implementation from Huggingface. We use the default sampling generation strategy with default hyperparameters, where the temperature is set as 0.6 and top_p is 0.9. About the meaning of these settings please refer to Llama technique report (Meta, 2024a) and model cards (Meta, 2024b).

To train these models, we use the AdamW optimizer (Loshchilov & Hutter, 2019) with a learning rate of $5e-5$, weight decay of 0.01, and batch sizes of 256, 64, and 32 for 0.1B, 0.9B, and 3B models, respectively. Other optimizer settings follow the default values in the Transformers library. We sample $1e7$ samples for *each* length (where feasible) and concatenate them into a single training

set. Models are trained for one epoch using a cosine decay learning rate scheduler, and the best checkpoint on validation data is reported.

Our experiments were conducted on a cluster equipped with Nvidia A800 GPUs (80GB memory). Training a 100M model takes 5–8 hours, a 1B model approximately 1 day, and a 3B model around 2 days on a single A800 GPU. Finetuning a pretrained model typically takes about 1 day.

Table 11: Detailed model settings for experiments.

parameter size	num hidden layers	hidden size	intermediate size	num attention heads	num KV heads
100M	8	1024	3584	8	2
0.9B	16	2048	7168	16	4
3.0B	24	3072	10752	24	6

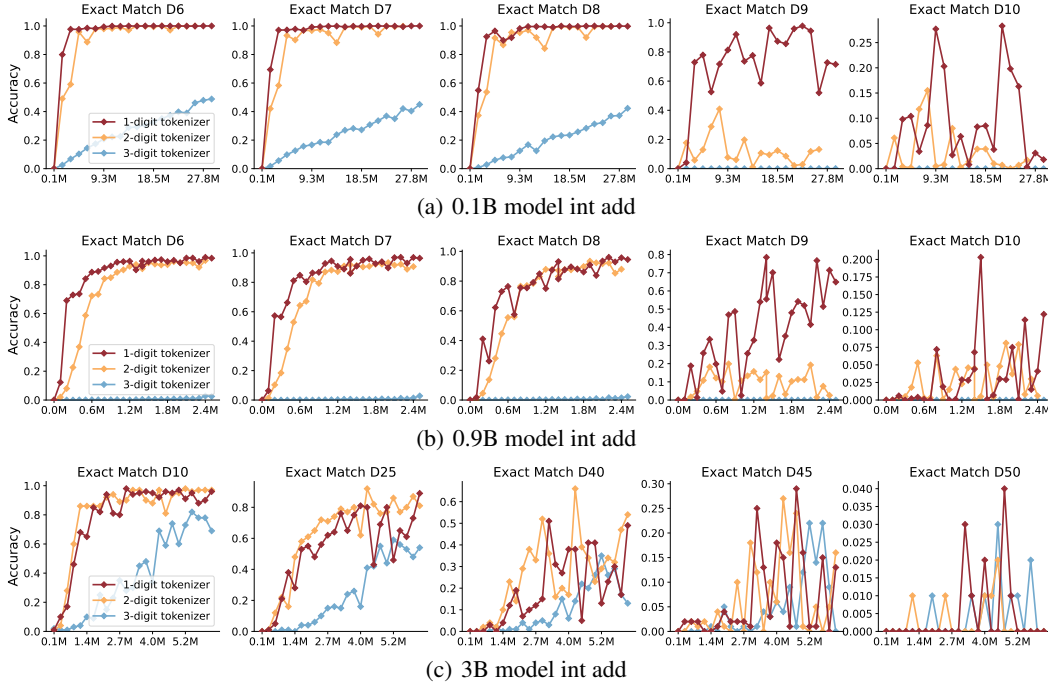


Figure 9: Accuracy of models of 0.1B, 0.9B and 3B parameters trained with 1-3 digit tokenizer on the task of integer addition. X-axis is the number of seen training samples.

A.4.2 TOKENIZATION

We experiment on models of 3 different size, including 0.1B, 0.9B and 3B. For the 0.1B and 0.9B models, we train them on integer addition of 1-8 digits; for the 3B model, we train it on the same task of 1-40 digits.

Figure 9 illustrates the in-domain performance of these three models in the first three columns and their out-of-domain (OOD) performance in the last two columns. Here we use the exact match metric. In our experiments of the 0.1B and 0.9B models, the one-digit and the two-digit tokenizer demonstrate comparable performance in the in-domain test, while the one-digit tokenizer exceeds the others to a large extent in length generalization. In contrast, the three-digit tokenizer exhibits poor performance in both in-domain and out-of-domain evaluations. Tokenizers with an increasing number of digits significantly hinder subbillion models’ NUPA. In the experiments of the 3B model, the two-digit tokenizer matches the one-digit tokenizer in both in-domain and OOD performance. In addition, the three-digit tokenizer shows the potential in length generalization for the first time, yet its performance remains inferior to that of the smaller tokenizers. This indicates that scaling up the model size

indeed alleviate the challenges in developing NUPA caused by larger tokenizers. Nevertheless, larger tokenizers do not present any distinct benefits in either in-domain or out-of-domain generalization in both small and large models.

We report the results according to different metrics from Figure 4 including digit match and dlength in Figure 10 and Figure 11.

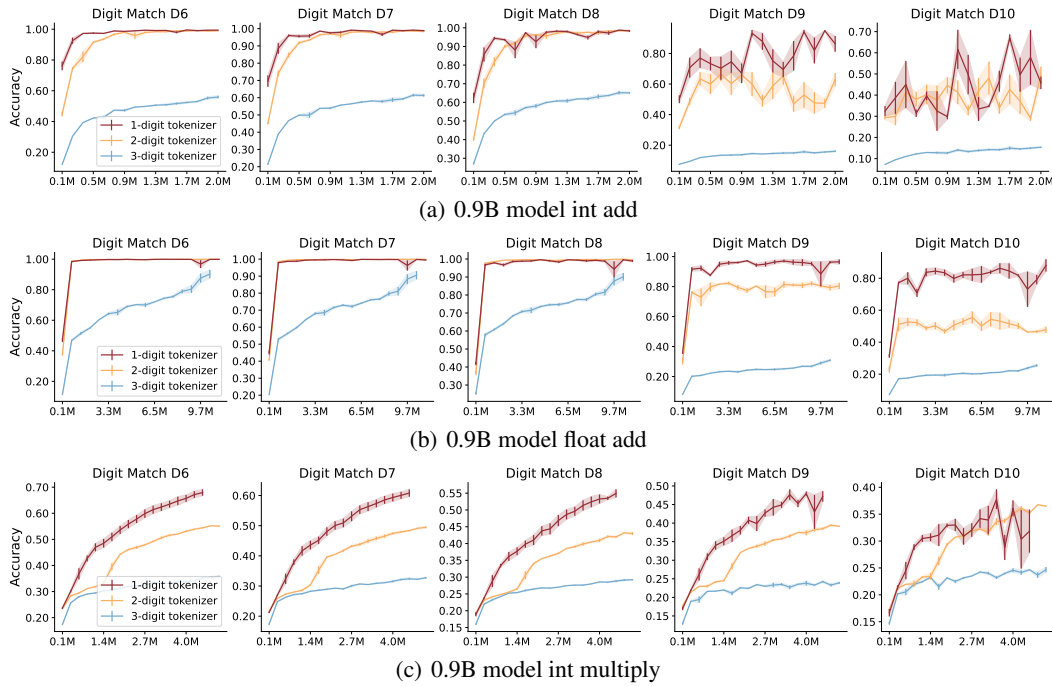


Figure 10: Accuracy of 0.9B models trained with 1-3 digit tokenizer on three tasks of integer addition, float addition and integer multiplication according to **digit match**. X-axis is the number of seen training samples.

Random tokenizer Introduced as “sub-word regularization” by Kudo (2018); Provilkov et al. (2020), the random tokenizer splits words like "Hello world" into variable tokens such as "He/llo/world" or "Hell/o/world". Though not widely used in LLMs, Sathe et al. (2024) found that it enhances reasoning by introducing variability in generation path. Inspired by this, we apply this to the numbers, segmenting numbers into tokens with lengths randomly chosen between 1 and a predefined maximum, instead of using greedy left-to-right segmentation.